

Mnożenie macierzy. Znajdowanie pary najbliższych punktów – Wykład

11 marca 2026

1 Mnożenie macierzy

A, B – macierze $n \times n$. $A \cdot B = C = ?$

Najpopularniejszy sposób mnożenia dwóch macierzy $n \times n$ działa w czasie $O(n^3)$. Musimy policzyć $O(n^2)$ wartości, obliczenie każdej zajmuje $O(n)$ czasu.

1.1 Algorytm Strassena

Zakładamy, że n jest potęgą 2 dla przejrzystości.

Dzielimy A i B na 4 macierze $\frac{n}{2} \times \frac{n}{2}$:

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, \quad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$
$$A \cdot B = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \cdot \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = C$$

gdzie:

$$C_{11} = A_{11} \cdot B_{11} + A_{12} \cdot B_{21}$$

$$C_{12} = A_{11} \cdot B_{12} + A_{12} \cdot B_{22}$$

$$C_{21} = A_{21} \cdot B_{11} + A_{22} \cdot B_{21}$$

$$C_{22} = A_{21} \cdot B_{12} + A_{22} \cdot B_{22}$$

Zamieniamy 1 mnożenie macierzy $n \times n$ na 8 mnożeń macierzy $\frac{n}{2} \times \frac{n}{2}$ + podział i dodawanie $\Theta(n^2)$.

Stąd mamy zależność rekurencyjną:

$$T(n) = 8 \cdot T\left(\frac{n}{2}\right) + O(n^2), \quad a = 8, \quad b = 2, \quad k = 2, \quad 8 > 2^2$$

$$T(n) = \Theta\left(n^{\log_2 8}\right) = \Theta(n^3)$$

Strassen zauważył, że jeśli obliczymy:

$$M_1 = (A_{12} - A_{22})(B_{21} + B_{22})$$

$$M_2 = (A_{11} + A_{22})(B_{11} + B_{22})$$

$$M_3 = (A_{11} - A_{21})(B_{11} + B_{12})$$

$$M_4 = (A_{11} + A_{12}) \cdot B_{22}$$

$$M_5 = A_{11} \cdot (B_{12} - B_{22})$$

$$M_6 = A_{22} \cdot (B_{21} - B_{11})$$

$$M_7 = (A_{21} + A_{22}) \cdot B_{11}$$

to

$$C_{11} = M_1 + M_2 - M_4 + M_6$$

$$C_{12} = M_4 + M_5$$

$$C_{21} = M_6 + M_7$$

$$C_{22} = M_2 - M_3 + M_5 - M_7$$

Zatem

$$T(n) = 7 \cdot T\left(\frac{n}{2}\right) + \Theta(n^2), \quad a = 7, \quad b = 2, \quad k = 2, \quad 7 > 2^2$$

$$T(n) = \Theta\left(n^{\log_2 7}\right) = \Theta(n^{2,81})$$

1.2 Własności optymalnej podstruktury

Optymalne rozwiązanie problemu jest funkcją optymalnych rozwiązań podproblemów (optymalne rozwiązanie problemu można skonstruować z optymalnych rozwiązań podproblemów).

Żeby metoda dziel i zwyciężaj działała dobrze, muszą być spełnione dwa warunki (przez problem i algorytm):

1. własności optymalnej podstawy,
2. podproblemy są rozłączne.

2 Znajdowanie pary najbliższych punktów

Q – zbiór $n \geq 2$ punktów na płaszczyźnie.

$$p_1 = (x_1, y_1), \quad p_2 = (x_2, y_2).$$

$$d(p_1, p_2) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2} \quad \text{– odległość euklidesowa}$$

Problem: Znaleźć parę punktów z Q w najmniejszej odległości.

Algorytm siłowy (brute force), który sprawdza wszystkie pary punktów, działa w czasie $O(n^2)$, bo mamy $\binom{n}{2} = \frac{n(n-1)}{2} = O(n^2)$ par punktów.

Podamy algorytm typu “dziel i zwyciężaj”, który działa w czasie $O(n \log n)$.

2.1 Opis algorytmu

W każdym wywołaniu rekurencyjnym wejście składa się ze zbioru $P \subseteq Q$ i dwóch tablic X, Y , które zawierają wszystkie punkty z P .

W tablicy X punkty są posortowane w porządku niemalejącym względem pierwszej współrzędnej, a w tablicy Y punkty są posortowane w porządku niemalejącym względem drugiej współrzędnej.

Jeśli $|P| \leq 3$, to sprawdzamy wszystkie pary.

Dla $|P| > 3$:

Dziel: Dzielimy punkty z P pionową prostą l na dwa podzbiory P_L i P_R , na lewo i prawo od l , tak że $P_L = \lceil \frac{|P|}{2} \rceil$ i $P_R = \lfloor \frac{|P|}{2} \rfloor$.

Dzielimy wtedy X na X_L i X_R zawierające punkty odpowiednio z P_L i P_R , posortowane w porządku niemalejącym pierwszej współrzędnej. Podobnie dzielimy Y na Y_L i Y_R zawierające punkty z P_L i P_R , posortowane w porządku niemalejącym drugiej współrzędnej.

Zwyciężaj: Wywołujemy rekurencyjnie algorytm dla P_L, X_L, Y_L i P_R, X_R, Y_R . Algorytm znajduje pary najbliższych punktów w P_L i P_R . Niech δ_L i δ_R będą najmniejszymi odległościami znalezionymi przez algorytm dla P_L i P_R odpowiednio.

Niech $\delta = \min(\delta_L, \delta_R)$.

Połącz: Para najbliższych punktów w P składa się z dwóch punktów z P_L , lub dwóch punktów z P_R , lub jednego z P_L i jednego z P_R .

Musimy sprawdzić, czy istnieje para trzeciego typu w odległości mniejszej niż δ . Jeśli taka para istnieje, wtedy oba jej punkty są w pionowym pasie o szerokości 2δ wokół l .

W celu sprawdzenia, czy taka para istnieje, wykonujemy następujące czynności:

1. Tworzymy tablicę Y' otrzymaną z Y poprzez usunięcie punktów spoza pasa. Y' jest posortowany w porządku niemalejącym względem drugiej współrzędnej.
2. Dla każdego punktu p z Y' sprawdzamy odległość między p a p' dla 7 kolejnych punktów p' z Y' .

Algorytm oblicza najmniejszą odległość δ' , zapamiętuje ją i odpowiednią parę punktów.

3. Jeśli $\delta' < \delta$, to pionowy pas zawiera parę punktów o mniejszej odległości niż pary znalezione w wywołaniach rekurencyjnych. Algorytm zwraca δ' i odpowiednią parę punktów. W przeciwnym przypadku algorytm zwraca δ i odpowiednią parę punktów.

2.2 Analiza poprawności

Jedyna rzecz, jaka nie jest oczywista, jest fakt, że sprawdzamy tylko 7 następnych punktów po p w Y' w fazie Połącz.

Dowód. Przyjmij, że p i q są parą punktów w najmniejszej odległości w P i $d(p, q) = \delta' < \delta$. Niech $p = (x_p, y_p)$, $q = (x_q, y_q)$ i $y_p \leq y_q$.

Pokażemy, że algorytm znajduje tę parę punktów.

Przypuśćmy, że to nieprawda. Wtedy istnieje co najmniej 7 punktów pomiędzy p i q w Y' . Niech $r = (x_r, y_r)$ będzie jednym z nich. Skoro $d(p, q) < \delta$, to

$$y_q - y_p = |y_q - y_p| \leq \sqrt{(x_p - x_q)^2 + (y_p - y_q)^2} = d(p, q) < \delta$$

Ponadto $y_p \leq y_r \leq y_q$, więc $y_r - y_p \leq y_q - y_p < \delta$. Zatem jest to co najmniej 9 punktów z P w prostokącie $2\delta \times \delta$ ograniczonym poziomymi liniami $y = y_p$ i $y = y_p + \delta$.

Wtedy w lewym kwadracie $\delta \times \delta$ lub w prawym kwadracie $\delta \times \delta$ jest przynajmniej 5 punktów z P . Przypuśćmy, że to lewy kwadrat.

Zatem istnieje kwadrat $\delta \times \delta$ z 5 punktami z P w P_L . Ale wtedy istnieją 2 punkty z P_L w odległości $< \delta$ w P_L , bo nie da się umieścić w kwadracie $\delta \times \delta$ 5 punktów, dla których odległość między dwoma dowolnymi jest $\geq \delta$ – sprzeczność (bo najmniejsze odległości w P_L to $\delta_L \geq \delta$).

Zatem wystarczy sprawdzić tylko 7 następnych punktów po p w Y' . □

2.3 Analiza złożoności czasowej

Na początku musimy posortować tablice X i Y – to zajmuje $O(n \log n)$.

Jeśli X i Y są posortowane, to podział X na posortowane X_L i X_R oraz podział Y na posortowane Y_L i Y_R kosztuje $O(n)$.

Mamy dwa wywołania rekurencyjne dla $\frac{n}{2}$ punktów i fazę Połącz. Tablicę Y' można otrzymać z Y w czasie $O(n)$, tak że Y' jest posortowana. Niech x_l będzie równaniem prostej l .

Skoro dla każdego z $O(n)$ punktów w Y' sprawdzamy odległość tylko do co najwyżej 7 punktów, to najmniejszą odległość δ' punktów z Y' znajdujemy w czasie $O(n)$.

$T(n)$ – złożoność czasowa algorytmu bez wstępnego sortowania:

$$T(n) = 2 \cdot T\left(\frac{n}{2}\right) + O(n)$$

$$a = 2, b = 2, k = 1, \quad 2 = 2^1 \implies T(n) = \Theta\left(n^{\log_2 2} \log n\right) = \Theta(n \log n)$$

Złożoność całego algorytmu wynosi $\Theta(n \log n) + \Theta(n \log n) = O(n \log n)$.